Comprehensive Guide to MySQL Stored Procedures and Triggers

Database programmability is essential for building robust, scalable, and secure applications. By moving business logic and data manipulation rules into the database itself, developers can ensure consistency across multiple applications and reduce network overhead. Two of the most powerful features for database programmability in MySQL are **Stored Procedures** and **Triggers**.

1. Stored Procedures in MySQL

A Stored Procedure is a prepared SQL code module that you save in the database and can call repeatedly. Stored procedures accept input parameters, perform complex operations, execute SQL queries, and return output parameters or result sets.

1.1 Benefits of Stored Procedures

- **Reduced Network Traffic:** Instead of sending multiple long SQL statements from the application to the database, you send only a single short call statement.
- **Reusability:** Procedures can be invoked by multiple applications or different parts of the same application.
- **Security:** You can grant users permission to execute a stored procedure without granting them direct SELECT, INSERT, UPDATE, or DELETE privileges on the underlying tables.
- **Maintainability:** Business logic centralized in the database can be updated in one place without recompiling application code.

1.2 Syntax and Structure

To create a stored procedure in MySQL, you must redefine the standard statement delimiter (usually `;`) using the `DELIMITER` command so that the semicolon inside the procedure body is not interpreted by MySQL as the end of the `CREATE PROCEDURE` statement.

```
DELIMITER //

CREATE PROCEDURE GetEmployeeById(
    IN p_emp_id INT,
    OUT p_name VARCHAR(100),
    OUT p_salary DECIMAL(10,2)
)
BEGIN
    SELECT name, salary
    INTO p_name, p_salary
    FROM employees
    WHERE id = p_emp_id;
END //

DELIMITER ;
```

1.3 Invoking and Dropping Procedures

You invoke a stored procedure using the `CALL` statement:

```
CALL GetEmployeeById(101, @emp_name, @emp_salary);
SELECT @emp_name, @emp_salary;
```

To remove a stored procedure, use the `DROP PROCEDURE` statement:

```
DROP PROCEDURE IF EXISTS GetEmployeeById;
```

1.4 Control Flow Statements

Stored procedures support procedural programming constructs like variables, conditions, and loops:

Variables:

```
DECLARE v_tax_rate DECIMAL(3,2) DEFAULT 0.05;
```

IF-THEN-ELSE:

```
IF v_salary > 50000 THEN
    SET v_tax_rate = 0.20;
ELSE
    SET v_tax_rate = 0.10;
END IF;
```

WHILE Loop:

```
WHILE v_counter <= 10 DO
    -- statements
    SET v_counter = v_counter + 1;
END WHILE;
```

2. Triggers in MySQL

A database trigger is a named database object that is associated with a table and is activated when a particular event occurs for the table. Triggers are highly useful for maintaining audit trails, enforcing complex business rules, and automatically updating derived values.

> **Important Note on Triggers**
>
> Triggers are executed automatically by the database engine when an `INSERT`, `UPDATE`, or `DELETE` occurs. They cannot be called directly.

2.1 Trigger Timing and Events

A trigger can be defined to execute either **BEFORE** or **AFTER** an event. The events are `INSERT`, `UPDATE`, and `DELETE`.

- `BEFORE` triggers allow you to validate or modify values before they are written to the table.
- `AFTER` triggers are typically used for logging, auditing, or cascading changes to other tables.

2.2 Using `NEW` and `OLD` Modifiers

Inside a trigger, you can access the row columns affected by the triggering event using the `NEW` and `OLD` modifiers:

- `NEW.column_name` holds the new row value for `INSERT` or `UPDATE` operations.
- `OLD.column_name` holds the existing row value before it is updated or deleted in `UPDATE` or `DELETE` operations.

2.3 Syntax and Example

Below is an example of an `AFTER UPDATE` trigger that records changes made to an employee's salary in an audit table.

```
DELIMITER //

CREATE TRIGGER after_employee_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit (employee_id, old_salary, new_salary, changed_at)
        VALUES (OLD.id, OLD.salary, NEW.salary, NOW());
    END IF;
END //

DELIMITER ;
```

2.4 Viewing and Dropping Triggers

To view all triggers in the current database:

```
SHOW TRIGGERS;
```

To remove a trigger:

```
DROP TRIGGER IF EXISTS after_employee_update;
```

3. Comparison and Best Practices

3.1 Key Differences

- **Invocation:** Stored procedures must be explicitly invoked via `CALL`. Triggers execute implicitly in response to `INSERT`, `UPDATE`, or `DELETE` events.
- **Parameters:** Stored procedures accept input and output parameters. Triggers do not accept parameters; they operate implicitly on row data via `NEW` and `OLD`.
- **Transactions:** Stored procedures can explicitly control transactions (`START TRANSACTION`, `COMMIT`, `ROLLBACK`). Triggers inherit the transaction context of the statement that activated them.

3.2 Best Practices

- **Keep Logic Simple:** Avoid complex business logic inside triggers, as they can silently impact performance and make debugging difficult.
- **Avoid Infinite Recursion:** Be careful with `AFTER UPDATE` or `BEFORE UPDATE` triggers that modify the same table, leading to cascading loops.
- **Error Handling:** Use standard SQLSTATE error handling within stored procedures to handle exceptions gracefully.
- **Performance Considerations:** Heavy operations inside triggers will slow down DML operations (inserts, updates, deletes) on large tables.